Implementação N.03: Caminho Mínimo com Menor Número de Arestas

Teoria dos Grafos e Computabilidade, PUC Minas, 2º semestre de 2026
Pedro Henrique Lopes de Melo

1. Implementação

O programa **CaminhoMinimo.java** recebe o nome de um arquivo com um grafo direcionado ponderado (pesos positivos) e um par de vértices (origem *s* e destino *t*). Ele exibe o comprimento do caminho mínimo, o número de arestas desse caminho e a lista de vértices que o compõem.

O método usado é o de **Dijkstra** na forma do slide 59 do material da disciplina: enquanto existir vértice imaturo, seleciona-se o vértice imaturo *v* que minimiza dist[ ], relaxa-se cada arco (*v*, *w*) tenso (dist[*w*] ← dist[*v*] + d_{vw}; pred[*w*] ← *v*) e declara-se *v* maduro.

Desempate pelo menor número de arestas. Além de dist[ ], cada vértice guarda arestas[ ], o número de arestas do melhor caminho encontrado até ele. O arco (*v*, *w*) passa a ser considerado tenso em dois casos: (i) dist[*v*] + d_{vw} < dist[*w*], como no método original; ou (ii) dist[*v*] + d_{vw} = dist[*w*] e arestas[*v*] + 1 < arestas[*w*]. A fila de prioridade ordena os vértices pelo par (dist, arestas). Como todos os pesos são positivos, todo predecessor de *w* em um caminho mínimo tem distância menor que a de *w* e, portanto, já está maduro quando *w* é retirado da fila; assim, arestas[*w*] já é o menor possível entre todos os caminhos mínimos.

Estruturas. O grafo é armazenado em lista de adjacência por meio de vetores (*forward star*): os sucessores do vértice *v* ocupam as posições de inicio[*v*] até a anterior a inicio[*v*+1] nos vetores destino[ ] e peso[ ]. A seleção do vértice imaturo de menor dist[ ] usa um heap binário (PriorityQueue do Java). A busca é iterativa (não recursiva) e para assim que o destino amadurece. Custo: O((n + m) log n) no pior caso.

Formato do arquivo: primeira linha com *n* e *m*; depois, uma aresta por linha no formato "origem destino peso", com os vértices numerados de 1 a *n* (o mesmo formato da Implementação N.01, acrescido do peso).

Exemplo de saída (grafo aleatório com 1.000 vértices, origem 655, destino 115):

```
Comprimento do caminho minimo: 13
Numero de arestas: 4
Vertices do caminho: 655 -> 656 -> 259 -> 787 -> 115
```

2. Metodologia dos testes

Foram usados dois tipos de grafos, gerados pelo programa **Gerador.java** (entregue junto), cada um em quatro tamanhos:

• **Aleatório:** *n* vértices e *m* = 5*n* arestas. As *n* primeiras formam o ciclo 1 → 2 → … → *n* → 1, o que garante que todo vértice alcança todos os outros; as demais ligam pares sorteados. Pesos inteiros sorteados de 1 a 10. Tamanhos: *n* = 1.000, 10.000, 100.000 e 1.000.000.

• **Grade:** *k* × *k* vértices, cada um com arcos para os vizinhos de cima, de baixo, da esquerda e da direita (*m* = 4*k*(*k*–1)). Pesos inteiros sorteados de 1 a 5. Tamanhos: *k* = 32, 100, 316 e 1.000 (*n* = 1.024, 10.000, 99.856 e 1.000.000).

Os pesos pequenos foram escolhidos de propósito: eles geram muitos caminhos mínimos de mesmo comprimento, que é exatamente o caso em que o desempate pelo menor número de arestas faz diferença. Em cada instância foram feitas 5 consultas com pares (origem, destino) sorteados (semente 42). Cada consulta é uma execução separada do programa.

Eficácia. Cada resposta foi conferida com uma implementação de referência independente (escrita em Python), que calcula, para o destino, a distância mínima, o menor e o maior número de arestas entre todos os caminhos mínimos e quantos caminhos mínimos existem. Uma resposta é considerada correta quando: o

caminho listado começa na origem e termina no destino; todos os seus arcos existem no grafo; a soma dos pesos é igual ao comprimento informado; o comprimento é igual à distância mínima da referência; e o número de arestas é igual ao menor número de arestas entre os caminhos mínimos.

Eficiência. O tempo da busca é medido dentro do programa com System.nanoTime() e não inclui a leitura do arquivo; o tempo total (leitura + montagem + busca) foi medido de fora, por execução. Ambiente: Intel Xeon 2,1 GHz (2 núcleos), 8 GB de RAM, OpenJDK 21, com 4 GB de memória reservados para a JVM.

3. Resultados

Tabela 1. Grafos aleatórios (médias das 5 consultas por instância)

n	m	Qtd.	Compr. médio	Arestas do escolhido	Arestas (mínimo mais longo)	Com empate	Busca média (ms)	Busca mín. e máx. (ms)	Total médio (s)	OK
1.000	5.000	5	14,8	5,0	5,2	1/5	3,1	1,9 a 5,2	0,21	5/5
10.000	50.000	5	23,8	8,4	9,4	1/5	14,8	5,4 a 22,8	0,39	5/5
100.000	500.000	5	28,2	8,4	9,0	2/5	91,1	14,0 a 121,2	1,09	5/5
1.000.000	5.000.000	5	37,8	12,2	13,0	2/5	1127,9	969,9 a 1236,8	7,36	5/5

Valores médios por consulta. "Arestas (mínimo mais longo)": maior número de arestas entre os caminhos mínimos, ou seja, o que um Dijkstra sem desempate poderia devolver no pior caso. "Com empate": consultas em que existe mais de um caminho mínimo.

Tabela 2. Grafos em grade (médias das 5 consultas por instância)

n	m	Qtd.	Compr. médio	Arestas do escolhido	Arestas (mínimo mais longo)	Com empate	Busca média (ms)	Busca mín. e máx. (ms)	Total médio (s)	OK
1.024	3.968	5	34,6	17,8	19,0	4/5	3,2	2,1 a 6,7	0,21	5/5
10.000	39.600	5	120,0	67,2	67,2	3/5	11,6	7,5 a 15,7	0,39	5/5
99.856	398.160	5	490,6	262,4	268,8	5/5	56,7	30,3 a 83,0	0,91	5/5
1.000.000	3.996.000	5	1327,8	741,4	758,2	5/5	253,8	64,0 a 401,0	5,09	5/5

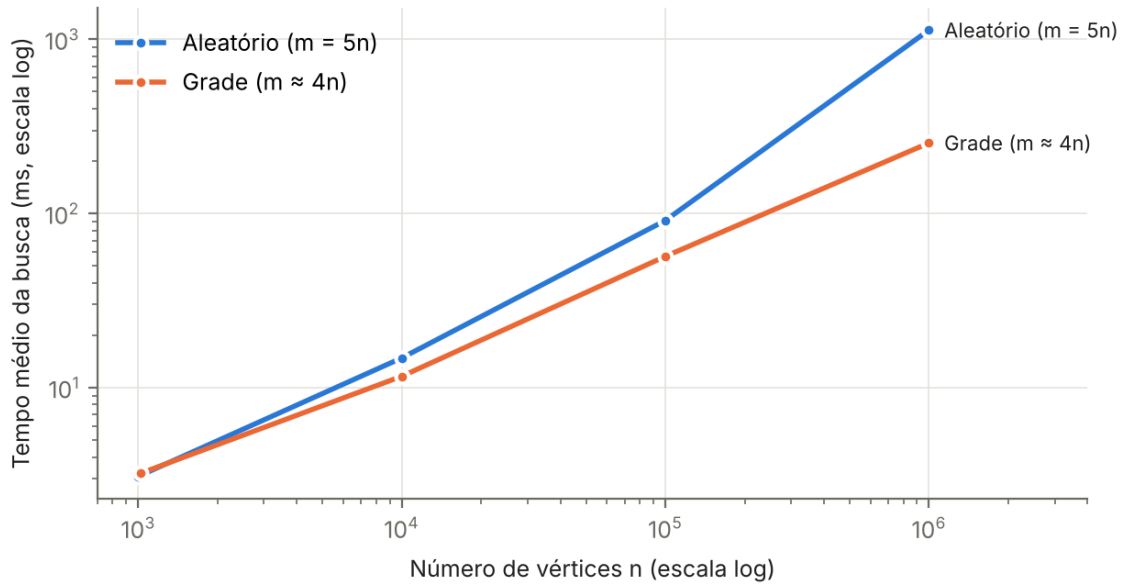


Figura 1. Tempo médio da busca em função do número de vértices (escalas logarítmicas).

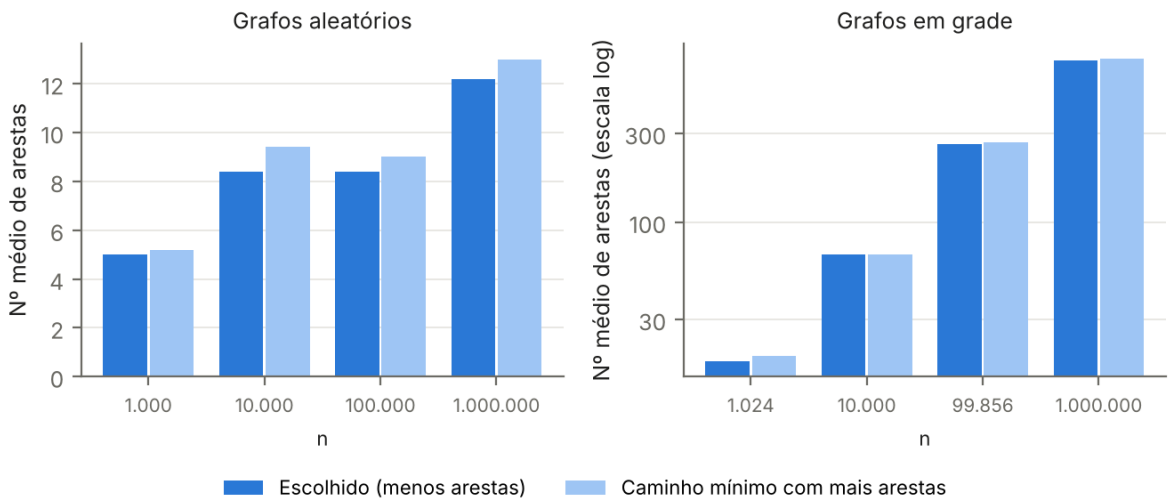


Figura 2. Número médio de arestas do caminho escolhido comparado ao caminho mínimo com mais arestas (mesmo comprimento).

4. Análise

Eficácia. As 40 de 40 consultas foram corretas: em todas, o caminho devolvido é válido, tem o comprimento mínimo e usa o menor número de arestas possível. Em 23 das 40 consultas havia mais de um caminho mínimo, e em 18 delas esses caminhos tinham números de arestas diferentes, ou seja, o desempate foi necessário. O efeito é muito maior nas grades: com pesos de 1 a 5 e muitos caminhos paralelos, o número de caminhos mínimos chega a ordens de grandeza de 10^{16} (ver apêndice), e na grade de 1.000.000 de vértices o caminho escolhido tem em média 16,8 arestas a menos que o caminho mínimo mais longo. Nos grafos aleatórios os empates são mais raros, porque a quantidade de caminhos curtos alternativos entre dois vértices sorteados é pequena.

Eficiência. Ao multiplicar *n* por 1.000, o tempo médio da busca cresceu cerca de 364 vezes nos grafos aleatórios e 78 vezes nas grades, compatível com O((n + m) log n). O tempo varia bastante entre consultas da mesma instância porque a busca para quando o destino amadurece: um destino próximo da origem é encontrado depois de explorar poucos vértices. Isso aparece com mais força nas grades, onde a distância entre os vértices sorteados varia muito, e explica por que as grades, mesmo com tamanho parecido, foram em média mais rápidas que os grafos aleatórios. Nas instâncias grandes, a leitura do arquivo com Scanner domina o tempo total (cerca de 7,4 s no aleatório e 5,1 s na grade de 1.000.000 de vértices, contra cerca de 1,1 s e 0,3 s de busca). As medidas de tempos pequenos (instâncias de 1.000 vértices) também incluem o aquecimento da JVM.

5. Como executar

```
javac CaminhoMinimo.java Gerador.java
java Gerador (tipo, tamanho, semente e nome do arquivo)
java CaminhoMinimo (nome do arquivo, origem e destino)
```

Apêndice: Resultado de cada consulta

Grafos aleatórios

n	Origem	Destino	Comprimento	Arestas do escolhido	Arestas (mínimo mais longo)	Nº de caminhos mínimos	Busca (ms)	Correto
1.000	655	115	13	4	4	1	1,9	sim
1.000	26	760	15	4	5	2	2,3	sim
1.000	282	251	10	5	5	1	3,1	sim
1.000	229	143	14	5	5	1	5,2	sim
1.000	755	105	22	7	7	1	3,0	sim
10.000	1825	410	18	5	5	1	5,4	sim
10.000	4507	4013	22	10	10	1	10,0	sim
10.000	3658	2287	23	12	12	1	22,8	sim
10.000	1680	8936	25	6	11	2	19,1	sim
10.000	1425	9675	31	9	9	1	16,8	sim
100.000	83811	14593	29	6	8	2	121,2	sim
100.000	3279	97197	29	11	11	1	110,8	sim
100.000	36049	32099	28	12	12	1	92,8	sim
100.000	29257	18290	21	4	4	1	14,0	sim
100.000	96531	13435	34	9	10	2	117,1	sim
1.000.000	670488	116740	37	13	14	2	1192,6	sim
1.000.000	26226	777573	36	14	14	1	969,9	sim
1.000.000	288390	256788	38	11	14	3	1236,8	sim
1.000.000	234054	146317	40	17	17	1	1058,3	sim
1.000.000	772247	107474	38	6	6	1	1181,9	sim

Grafos em grade

n	Origem	Destino	Comprimento	Arestas do escolhido	Arestas (mínimo mais longo)	Nº de caminhos mínimos	Busca (ms)	Correto
1.024	229	52	39	21	21	2	2,4	sim
1.024	564	502	11	4	4	5	2,1	sim
1.024	458	286	56	28	32	38	6,7	sim
1.024	210	179	6	2	2	1	2,4	sim
1.024	865	66	61	34	36	2	2,5	sim
10.000	1825	410	52	29	29	4	9,8	sim
10.000	4507	4013	20	11	11	1	7,5	sim
10.000	3658	2287	74	43	43	1	9,9	sim
10.000	1680	8936	217	119	119	24	15,3	sim
10.000	1425	9675	237	134	134	216	15,7	sim
99.856	83811	14593	504	269	277	192	62,8	sim
99.856	3279	97197	715	379	389	5.241.024	83,0	sim
99.856	36049	32099	373	191	193	48	43,2	sim
99.856	29257	18290	237	130	134	240	30,3	sim
99.856	96531	13435	624	343	351	23.040	64,0	sim
1.000.000	670488	116740	1444	822	848	3,62e+11	401,0	sim
1.000.000	26226	777573	1982	1128	1142	1,70e+16	322,2	sim
1.000.000	288390	256788	899	484	488	155.520	183,3	sim
1.000.000	234054	146317	648	365	377	425.088	64,0	sim
1.000.000	772247	107474	1666	908	936	5,31e+15	298,6	sim